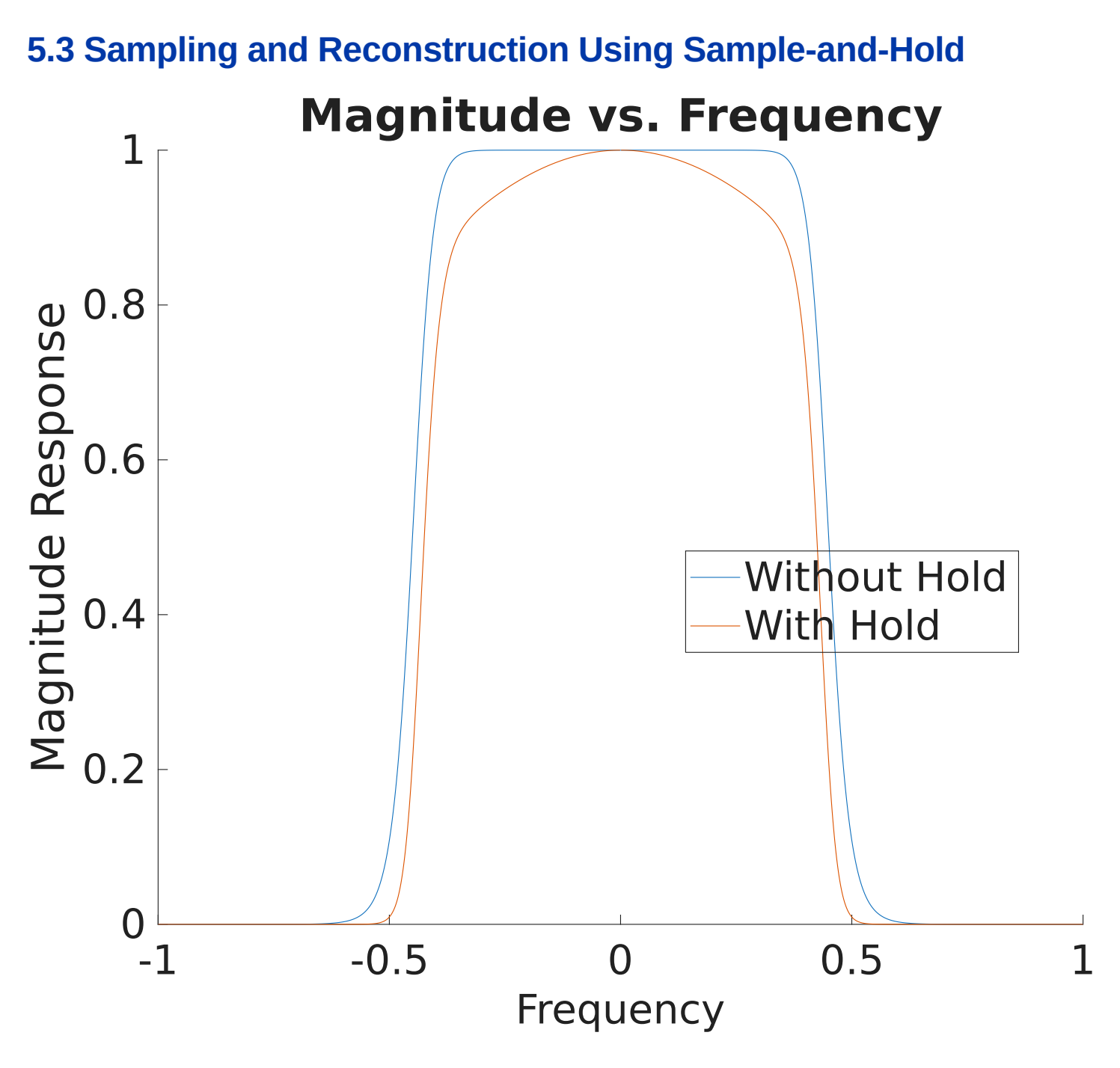
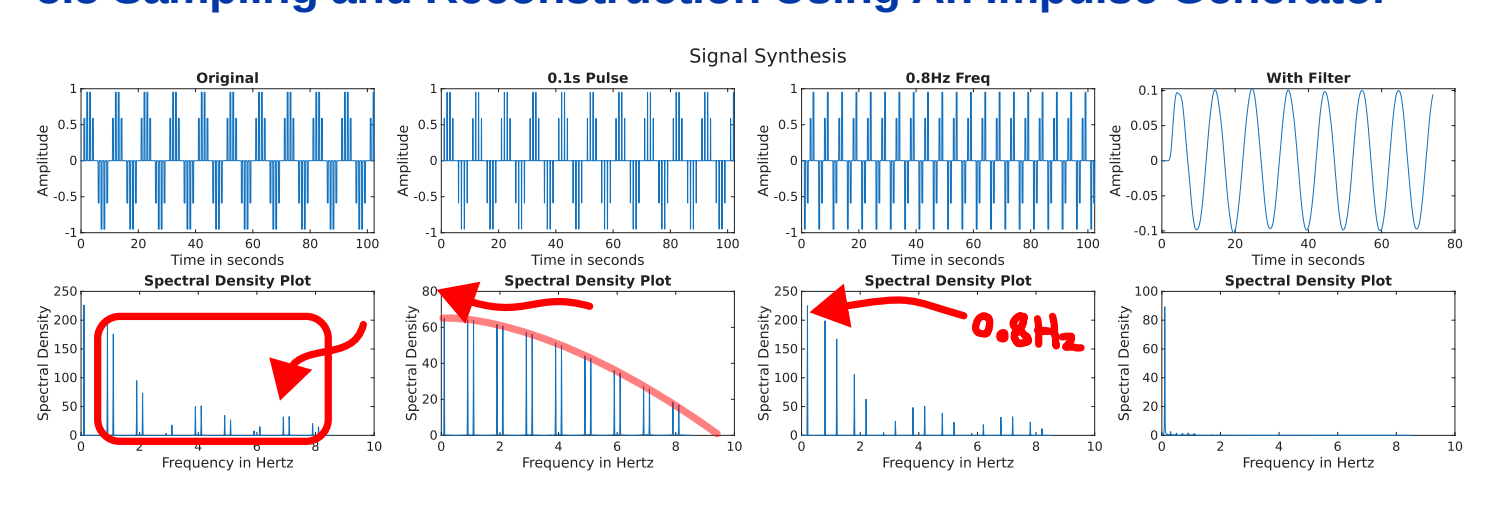
Lab 4

5.3 Sampling and Reconstruction Using Sample-and-Hold



The frequency response of holdless system is significantly more consistent from the 0 to 0.35Hz range, allowing for more equal audio response if it were a CD player. The response with hold, however, is not consistent within this range, nor consistently flat at any range, making it a bad reproduction of audio had this be a CD player.

5.5 Sampling and Reconstruction Using An Impulse Generator

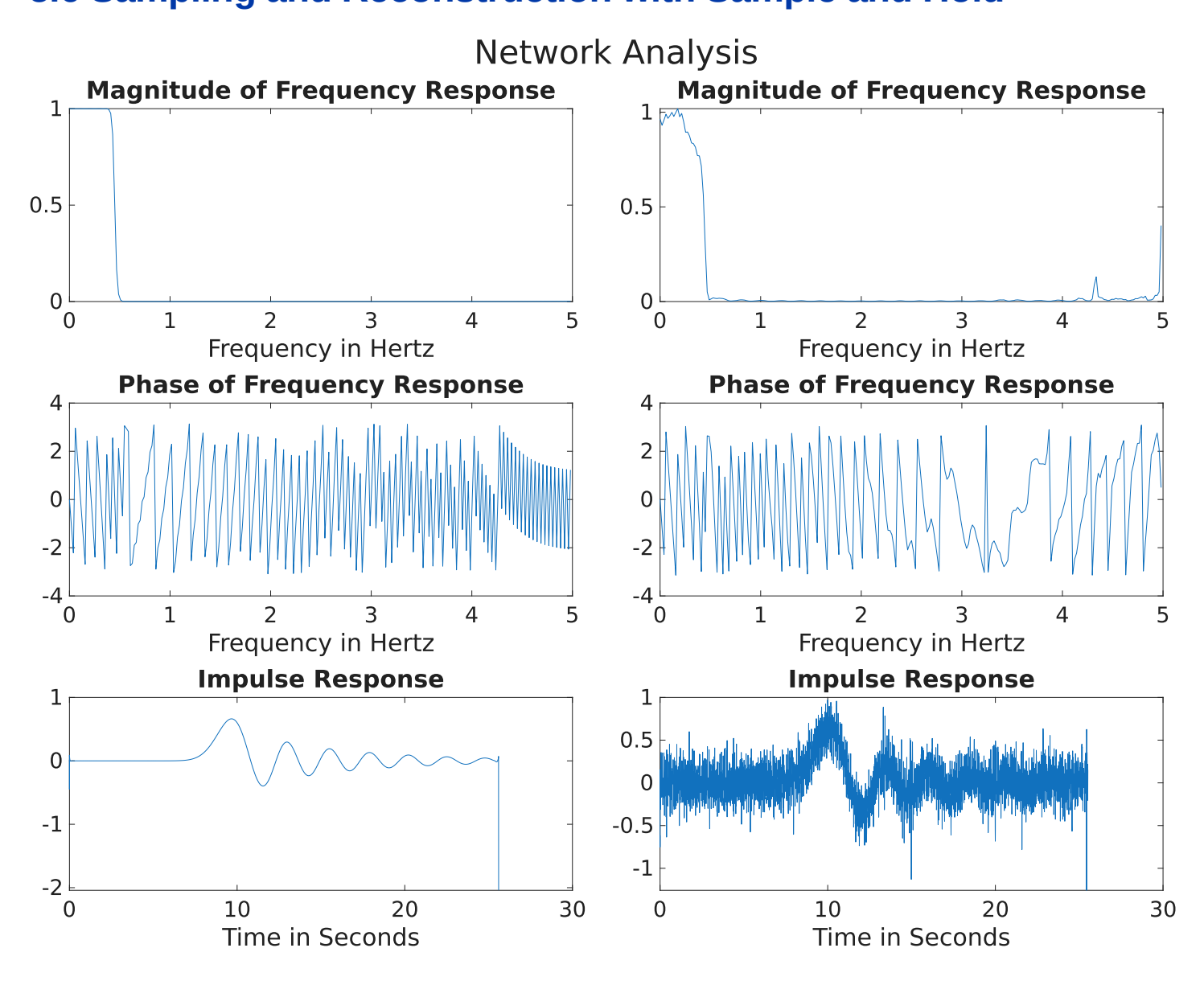


Since the pulse frequency is now 0.1s instead of 0.3s, the bounce seen in the spectral density plot has a much lower frequency. The bounce period will be equivalent to the original frequency multiplied by the pulse frequency, meaning when our pulse frequency increased to 10Hz, the bounce period increases, as seen in the plot.

The lowest frequency is now a result of the Doppler effect of our sampling frequency and source frequency, making the lowest frequency the a combination of the two original frequencies, at 0.2Hz which is 1-0.8Hz.

We filtered out all the unwanted frequencies, leaving us with the original signal that we wanted pre-transmission.

5.6 Sampling and Reconstruction with Sample and Hold

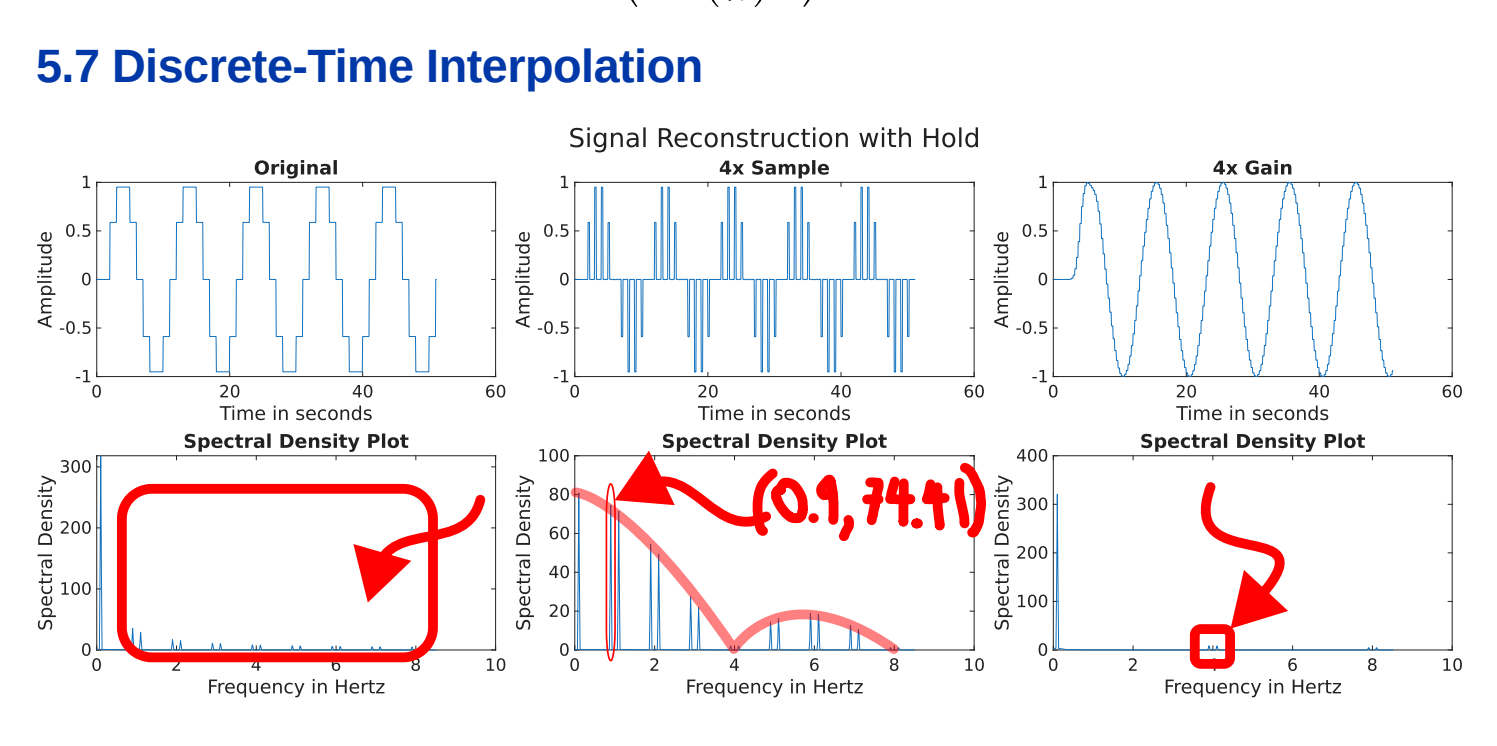


The frequency response of the delayed signal has a much more rounded falloff as compared to that without the delay. We likely see this falloff since we are sampling after a certain amount of time, meaning we might hit an off peak delay of each frequency, underrepresenting its magnitude in the final output.

The equation was found earlier in section 5.3 and is as such:

$$|H(f)| = \left(\frac{1}{1+\left(\frac{f}{f_c}\right)^N}\right)^2 \left|\text{sinc}\left(\frac{f}{f_s}\right)\right|$$

5.7 Discrete-Time Interpolation



The envelope of the second plot bounces with a period of 4 Hz, with the first bounce at 4Hz and a second at 8Hz.

The cutoff frequency we used $\frac{\pi}{4}$, as we were upsampling by 4, so we needed to cut off anything at or above 0.9Hz, so we chose 0.25Hz to cleanly fully remove all unnecessary frequencies. Interpolation allows us to have a higher resolution output signal as compared to the lower resolution recieving capabilities.

5.8 Discrete-Time Decimation

The code for this section may be found below within the file title `q58.m`. Yes, sig1 has significantly more distortion than sig2, as we threw away half of our samples, leading to sharp changes in pressure and the introduction of high frequency noises when high notes are played in the original recording. Sig2 did not have this issue as we deal directly with frequencies instead of samples, allowing us to maintain a similar frequency response to the original sample.

Code

`helpers.m`

```
% A function to help set our defaults for all plots
function [fontSize, export] = helpers(f, varargin)
    p = inputParser;

    % Define defaults
    addParameter(p, 'w', 1000);
    addParameter(p, 'h', 800);
    parse(p, varargin{:});

    % Disables LaTeX rendering, as it is not supported by the export function
well
    set(f,'defaultTextInterpreter','none')
    set(f,'defaultAxesTickLabelInterpreter','none')
    set(f,'defaultLegendInterpreter','none')
    % Increase Default font size
    defaultFontSize = 25;
    set(f, 'defaultTextFontSize', defaultFontSize);
    set(f, 'defaultAxesFontSize', defaultFontSize);
    set(f, 'defaultLegendFontSize', defaultFontSize);

    % Set Font
    set(f, 'DefaultAxesFontName', 'Arial');
    set(f, 'DefaultTextFontName', 'Arial');
    set(f, 'DefaultLegendFontName', 'Arial');
    set(f, 'DefaultColorbarFontName', 'Arial');

    % Set clear background
    set(f, 'DefaultFigureColor', 'none');
    set(f, 'DefaultAxesColor', 'none');
    set(f, 'DefaultLegendColor', 'none');
    set(f, 'DefaultFigureInvertHardcopy', 'off');

    % Set the default layout to "loose"
    set(f, 'DefaultTiledLayoutTileSpacing', 'loose');
    set(f, 'DefaultTiledLayoutPadding', 'loose');

    % Increase Canvas size
    set(f, 'Position', [0 0 p.Results.w p.Results.h]);

    % Set text alignment properties
    set(f, 'DefaultAxesTitleHorizontalAlignment', 'center');
    set(f, 'DefaultTextHorizontalAlignment', 'center');

    % Disable slow output warning
    warning('off', 'MATLAB:print:ContentTypeImageSuggested')

    % Set high quality output by default
    set(f, 'DefaultFigureRenderer', 'painters');
    f.Visible = "on";
    % Return our font size
    fontSize = defaultFontSize;
    export = @(fp) exportgraphics(f, fp, ContentType='vector',
BackgroundColor='none');
end
```

`plotNet.m`

```
% Plot Network Analysis given axes, time, input samples, output samples, and
number of samples to plot
function plotNet(pl, t, ins, outs, N)
    % Find total samples and Ts
    Nn = numel(ins);
    Ts = (t(Nn) - t(1))/Nn;

    % Calculate FFTs of inputs and outputs
    outsf = fft(outs);
    insf = fft(ins);

    % Calculate our transfer function
    transfer = outsf./insf;
    Fs = 1/Ts;
    % Resample our x-axis for frequency
    f = (0:N-1) * (Fs/Nn);

    % Plot Magnitude and Phase of the response
    pls = nexttile(pl);
    plot(pls, f(1:(N/2)), abs( transfer(1:(N/2))) );
    xlabel(pls, 'Frequency in Hertz');
    title(pls, 'Magnitude of Frequency Response');

    pls = nexttile(pl);
    plot(pls, f(1:(N/2)), angle( transfer(1:(N/2))) );
    xlabel(pls, 'Frequency in Hertz');
    title(pls, 'Phase of Frequency Response');

    % Plot our Impulse response using ifft
    pls = nexttile(pl);
    h = ifft(transfer);
    h=h/Ts;
    plot(pls, t(1:(round(Nn/2))), real(h(1:(round(Nn/2)))) );
    xlabel(pls, 'Time in Seconds');
    title(pls, 'Impulse Response');
end
```

`plotSpec.m`

```
% Plot Spectran Analysis given axes, time, signal, number of items to plot, and
title
function plotSpec(pl, t, signal, N, ti)
    % Find total samples and Ts
    Nn = numel(signal);
    Ts = (t(Nn) - t(1))/Nn;

    % Plot Signal itself
    pls = nexttile(pl);
    plot(pls, t(1:2*N), signal(1:2*N));
    xlabel(pls, 'Time in seconds');
    ylabel(pls, 'Amplitude');
    title(pls, ti);

    % Calculate the amplitude of the FFT
    X = abs(fft( hamming(Nn).*signal ));
    f = (1/Ts)*((1:Nn)-1)./Nn;

    % Plot FFT analysis
    pls = nexttile(pl);
    plot(pls, f(1:N),X(1:N));
    xlabel(pls, 'Frequency in Hertz');
    ylabel(pls, 'Spectral Density');
    title(pls, 'Spectral Density Plot');
end
```

`q53.m`

```
% Define our plot functions
f1 = @(ts, fc, N) @(x) 1 ./ (1 + (x./fc).^N);
f2 = @(ts, fc, N) @(x) (1 ./ (1 + (x./fc).^N)).^2 .* abs(sin(x./fc) ./
(x./fc));

% Determine parameters
ts = 1;
fc = 0.45;
N = 20;

% Evaluate functions
f1e = f1(ts, fc, N);
f2e = f2(ts, fc, N);

% Define domain
domain = -1:0.001:1;

% Set up canvas
f = figure;
[ff, e] = helpers(f);

% Plot angle and amplitude
hold on
plot(domain, f1e(domain))
plot(domain, f2e(domain))
legend( ...
    'Without Hold', ...
    'With Hold', ...
    Location='best', ...
    FontSize=ff )
xlabel("Frequency", FontSize=ff)
ylabel("Magnitude Response")
title("Magnitude vs. Frequency")

% Export graphic
e('plots/1_q53.svg')
```

`q55.m`

```
% Add the simulink files to path
addpath("simu")

% The model we are working on
mdl = 'ImpulseGen';

% Load simulink model and turn on logging
system = load_system(mdl);
set_param(mdl, ReturnWorkspaceOutputs="on")

% Our output plot length
N = 1024;

% Define all values that change
names = {"Signal Generator", "Impulse Generator"};
values = {{"0.1", "0.3"}, {"0.1", "0.1"}, {"0.8", "0.3"}};
param = {"Frequency", "MaskParam2"};
titles = {"Original", "0.1s Pulse", "0.8Hz Freq"};

% Plot each spectral analysis
f = figure;
[ff, e] = helpers(f, w=4000, h=1200);
t = tiledlayout(2, 4, TileIndexing="columnmajor");

for value=1:numel(values)
    % Set values we care about
    for name=1:numel(names)
        set_param(sprintf("%s/%s", mdl, names{name}), param{name},
values{value}{name});
    end
    % Run simulation
    outs = sim(mdl);
    % Plot spectral analysis on our figure
    plotSpec(t, outs.simout.time, outs.simout.data, N, titles{value})
end

close_system(mdl, 0);
Simulink.sdi.clear;

% The second model we are working on
mdl = 'ImpulseGenFiltered';

% Load simulink model and turn on logging
system = load_system(mdl);
set_param(mdl, ReturnWorkspaceOutputs="on")

% Run simulation
outs = sim(mdl);
% Plot spectral analysis on our figure
plotSpec(t, outs.simout.time, outs.simout.data, N, "With Filter")

% Title toplevel
title(t, "Signal Synthesis", FontSize=ff*1.5);
% Export full graph
e("plots/2_q55.svg")

% Clean up
close_system(mdl, 0);
Simulink.sdi.clear;
```

`q56.m`

```
% Add the simulink files to path
addpath("simu")

% The model we are working on
mdl = 'Samp';

% Load simulink model and turn on logging
system = load_system(mdl);
set_param(mdl, ReturnWorkspaceOutputs="on")

% Our output plot length
N = 512;

% Plot each spectral analysis
f = figure;
[ff, e] = helpers(f, h=1500, w=2000);
t = tiledlayout(3, 2, TileIndexing="columnmajor");

% Run simulation
outs = sim(mdl);
% Plot spectral analysis on our figure
plotNet(t, outs.simout.time, outs.simin.data, outs.simout.data, N)

% Clean up
close_system(mdl, 0);
Simulink.sdi.clear;


% The second model we are working on
mdl = 'SampHold';

% Load simulink model and turn on logging
system = load_system(mdl);
set_param(mdl, ReturnWorkspaceOutputs="on")

% Run simulation
outs = sim(mdl);
% Plot spectral analysis on our figure
plotNet(t, outs.simout.time, outs.simin.data, outs.simout.data, N)

% Clean up
close_system(mdl, 0);
Simulink.sdi.clear;

% Title toplevel
title(t, "Network Analysis", FontSize=ff*1.5);
% Export full graph
e("plots/3_q56.svg")
```

`q57.m`

```
% Add the simulink files to path
addpath("simu")

% The model we are working on
mdl = 'Interp';

% Load simulink model and turn on logging
system = load_system(mdl);
set_param(mdl, ReturnWorkspaceOutputs="on")

% Our output plot length
N = 512;

% Define all values that change
names = {"Upsampler", "Discrete-time LP Filter", "Discrete-time LP Filter",
"Gain"};
values = {{"1", "1", "3.1415", "1"}, {"4", "0.25", "3.1415", "1"}, {"4",
"0.25", "3.1415/4", "4"}};
param = {"L", "MaskParam3", "MaskParam1", "Gain"};
titles = {"Original", "4x Sample", "4x Gain"};

% Plot each spectral analysis
f = figure;
[ff, e] = helpers(f, w=3000, h=1200);
t = tiledlayout(2, 3, TileIndexing="columnmajor");

for value=1:numel(values)
    % Set values we care about
    for name=1:numel(names)
        set_param(sprintf("%s/%s", mdl, names{name}), param{name},
values{value}{name});
    end
    % Run simulation
    outs = sim(mdl);
    % Plot spectral analysis on our figure
    plotSpec(t, outs.simout.time, outs.simout.data, N, titles{value})
end

% Clean up simulink
close_system(mdl, 0);
Simulink.sdi.clear;

% Title toplevel
title(t, "Signal Reconstruction with Hold", FontSize=ff*1.5);
% Export full graph
e("plots/4_q57.svg")
```

`q57.m`

```
% Define our systems
y36 = @(s) s - [0, s(1:numel(s)-1)];
y37 = @(s) reduce(s, @(p,n) 0.5.*p + n, 0);

% Read the audio
[ audio, Fs ] = audioread("res/music.au");

% Calculate signal 1 by naiively decimating
sig1 = audio(1:2:end);

% Calculate signal 2 by convolving then decimating
h = fir1(20, 0.5);
c = conv(audio, h);
sig2 = c(1:2:end);

% % Save the outputs to mp3 files
audiowrite("res/audio.mp3", audio, Fs);
audiowrite("res/sig1.mp3", sig1, Fs);
audiowrite("res/sig2.mp3", sig2, Fs);
```

`runall.m`

```
% Clean workspace
close all
clear all

warning('off', 'Simulink:SFunctions:ObsoleteMFileSFcn');

% Define working functions
functions = { @q53, @q55, @q56, @q57, @q58 };

% Clean after each function
for i=1:numel(functions)
    functions{i}();
    clearvars -except functions i
end

close all
clear all
```